Application development for Android

Jetpack Compose exercise

Dependency injection

Photos: gallery and camera

Permissions

Image processing libraries

Geolocation

Maps (Google Maps and Open Street Maps)

Contacts application

- Create an application to manage contacts
  - A contact will be defined by
    - 1st version: name, e-mail, phone and birthday (can be `null`)
    - 2nd version: additionally, add an image of the contact
    - 3rd version: include geolocation information about the last times a contact was met
  - The model data must be platform independent

Project organization

(Update Android Studio to the latest version!!)

- Create an *Android* project choosing "*Empty Activity*" for the first *Activity*
- Create a project packaging organization that allows to separate application concerns (1st version)
  - `<base package>`
    - `model`
      - The classes in this package cannot import platform-specific libraries
    - `ui`
      - `screens`
        - include all the composables which represent application screens
      - `theme`
        - already created with the project
        - include configurations on generic appearance characteristics
      - `viewmodels`
        - include all *`ViewModel`* classes
      - ***`MainActivity`*** class
        - already created with the project
    - ***`ContactsApp`*** class (extends *`Application`*)
      - Create this class and register it in the manifest file

Model

- Create a serializable class to represent a contact

```
class Contact(
    var name: String,
    var email: String,
    var phone: String,
    var birthday: Date?=null,
) : Serializable
```

Model

- Create a serializable class to represent the list of contacts

```
class ContactsList : Serializable {
    private val contacts = mutableListOf<Contact>()

    fun addContact(contact: Contact) {
        contacts.add(contact)
    }

    fun getContacts(): List<Contact> {
        return contacts.toList()
    }

}
```

Model

- Make it possible to sort the list by contact's name, taking into consideration the special characters of the current device language by using:

```
fun getContacts(): List<Contact> {
    val collator = Collator.getInstance(Locale.getDefault()).apply {
        strength = Collator.PRIMARY
    }

    return contacts.sortedWith { contact1, contact2 ->
        collator.compare(contact1.name, contact2.name)
    }
}
```

Model

- Add methods to save and restore the data model

```
class ContactsList : Serializable {
    /* ... */
    fun save(destination: OutputStream) {
        try {
            ObjectOutputStream(destination).use { oos ->
                oos.writeObject(this)
            }
        } catch (_: Exception) { }
    }
    companion object {
        fun load(source : InputStream) : ContactsList?  {
            try {
                ObjectInputStream(source).use { ois ->
                    return ois.readObject() as ContactsList
                }
            } catch (_: Exception){ return null }
        }
    }
}
```

Application data

- Create an instance of the model data in the application object
  - To avoid slow application startup, the instance of the model should be created only when it is first accessed
- Provide a method to save data

```
class ContactsApp : Application() {
    companion object {
        private const val DATAFILE = "contacts.bin"
    }
    private val _contactsList by lazy {
        try {
            openFileInput(DATAFILE)?.let { srcFile -> ContactsList.load(srcFile) }
        } catch (_: Exception) {
            null
        } ?: ContactsList()
    }
    val contactsList get() = _contactsList

    fun saveData() {
        try {
            contactsList.save(openFileOutput(DATAFILE, MODE_PRIVATE))
        } catch (_: Exception) {}
    }
}
```

View Model

- Create a `ViewModel` class that receives a data model instance via dependency injection

```
class ContactsViewModelFactory(
    private val contactsList : ContactsList
) : ViewModelProvider.Factory {

    @Suppress("UNCHECKED_CAST")
    override fun <T:ViewModel> create(modelClass: Class<T>): T {
        return ContactsViewModel(contactsList) as T
    }
}

class ContactsViewModel(
    val contactsList : ContactsList
) : ViewModel() {

}
```

View Model (Activity)

- Create an instance of the `ViewModel` for use within the `MainActivity` context
  - In the context of this application, only one instance of the `ViewModel` will be created, and it will be shared across multiple screens (*composables*)

```
class MainActivity : ComponentActivity() {
    private val app by lazy { application as ContactsApp }
    private val viewModel : ContactsViewModel
                        by viewModels { ContactsViewModelFactory(app.contactsList) }
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AMovContactsTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Text("Hello World", modifier = Modifier.padding(innerPadding))
                }
            }
        }
    }

    override fun onPause() {
        super.onPause()
        app.saveData()
    }
}
```

UI Interface

- The application should display a main screen with a list of contacts
  - This list should present each contact as a `Card`
  - By default, only the contact's name will be shown
    - Include an `IconButton` in the `TopBar` to switch between this view and an expanded view to show more of the contact's information

- When a `Card` is clicked, a new screen should display all the information related to the selected contact
  - On this screen, an *Edit* `IconButton` in the `TopBar` should allow the user to edit the contact's information

UI Interface (cont.)

- In the main screen, an *Add* `IconButton` should enable the user to create a new contact, using a new screen for this operation
  - Use the same screen for creating a new contact or editing an existing one
  - On this screen, a *Done* `IconButton` should confirm the changes, update the global contact list, and return the user to the main screen

- All screens should display a back navigation icon (back arrow) in the `TopBar`, except the main screen

Main Screen

- Create a *Composable* to represent the main screen: `MainScreen`
- This screen will act as the base for the multiple screens in the application, including:
  - The `Scaffold` composable
  - Management of the *Top Bar* icons
  - Navigation management between screens (*List*, *Show* and *Edit*)
- This composable receives by parameter the reference for the View Model

```
@Composable
fun MainScreen(
    viewModel: ContactsViewModelFactory,
    modifier: Modifier = Modifier
) {
    Scaffold(modifier = modifier.fillMaxSize()) {
      innerPadding ->
        Text( "Hello World", modifier = Modifier.padding(innerPadding))
    }
}
```

Navigation

- When managing the display of different screens within an *Activity*, two main components become essential: the `NavHostController` and the `NavHost`

- To access these and related navigation components, include the following dependency in the application module `build.gradle` file:

```
implementation("androidx.navigation:navigation-compose:2.8.4")
```

Navigation

- The `NavHostController` (a subclass of `NavController`) is the core element for managing *Navigation* within an application
  - Manages and remembers the current screen
  - Maintains the back stack of composable screens
  - Preserves the state of each screen in the navigation stack

- To create a `NavHostController` instance, use:

```
val navController = rememberNavController()
```

Navigation

- A `NavHost` links a `NavHostController` with a navigation graph, which defines the composable destinations and their routes
- Each destination in the navigation graph is associated with a unique route
- `NavHostController` provides several methods, e.g.:
  - `navigate("route")`
  - `navigateUp()`
  - `popBackStack()`
  - `popBackStack("route", inclusive : Boolean)`
  - `currentBackStackEntryAsState()`
  - `addOnDestinationChangedListener(listener)`, …
- When navigation occurs between composables, the `NavHost` automatically recomposes its content to reflect the new destination

```
NavHost(navController, startDestination = "list",...) {
    composable("list") { ListScreen(...) }
    composable("show") { ShowScreen(...) }
    ...
}
```

Main Screen

- Include the `NavHost` in the `MainScreen`
  - Apply the `Scaffold`'s `innerPadding` value to the `NavHost` composable

```
@Composable
fun MainScreen(
    viewModel: ContactsViewModel,
    navController: NavHostController = rememberNavController(),
    modifier: Modifier = Modifier
) {
    var currentScreen by navController.currentBackStackEntryAsState()
    Scaffold(modifier = modifier.fillMaxSize()) { innerPadding ->
        NavHost(
            navController = navController,
            startDestination = "list",
            modifier = Modifier.padding(innerPadding)
        ) {
            composable("list") { Text("List screen") }
            composable("show") { Text("Show screen") }
            composable("edit") { Text("Edit screen") }
        }
    }
}
```

Main Screen

- Add and configure the *Top Bar* to include the application title and the buttons for navigation and contextual actions related to each screen
  - Change the `startDestination` of the `NavHost` to test each screen *Top Bar* buttons, that will be included in the next steps

```
Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Contacts") },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = Color(0,224,255),
                    titleContentColor = Color(0,0,128),
                ),
                navigationIcon = {  },
                actions = {  },
            )
        },
        ...
```

Navigation Icon

- Include an *Arrow Back* `IconButton` to return to the previous screen
  - This icon cannot appear in the *Main Screen*
  - Create a state variable to be aware of changes to the current screen:

```
val currentScreen by navController.currentBackStackEntryAsState()
```

  - In the `TopAppBar` object:

```
navigationIcon = {
    if (currentScreen?.destination?.route != "list") {
        IconButton(
            onClick = { navController.navigateUp() }
        ) {
            Icon(
              Icons.AutoMirrored.Filled.ArrowBack,
              contentDescription = "Back"
            )
        }
    }
},
```

Action buttons

- Define the already mentioned action buttons to the *Top Bar*
  - *List Screen*: "expand/collapse" and "add"
  - *Show Screen*: "edit"
  - *Edit Screen*: "done/save"

```
actions = {
    when (currentScreen?.destination?.route) {
        "list"->{ ... }
        "show"->{ ... }
        "edit"->{ ... }
    }
}
```

Action buttons – List Screen

- Button to expand/collapse cards

```
IconButton(onClick = { }) {
    Icon(
        Icons.AutoMirrored.Filled.List,
        contentDescription = "Expand/Collapse"
    )
}
```

- Create a state variable to manage the expand mode and change its value with the previous button

```
var showExpanded by remember { mutableStateOf(false) }
```

Action buttons – List Screen

- Button to add a new contact

```
IconButton(
    onClick = { navController.navigate("edit") }
) {
    Icon(
        Icons.Filled.Add,
        contentDescription = "Add Contact"
    )
}
```

Action buttons – Show Screen

- Button to edit the selected contact

```
IconButton(
    onClick = { navController.navigate("edit") }
) {
    Icon(
        Icons.Filled.Edit,
        contentDescription = "Edit Contact"
    )
}
```

Action buttons – Edit Screen

- Button to confirm the changes and return to the *List Screen*

```
IconButton(
    onClick = {
        navController.popBackStack("list", inclusive=false)
    }
) {
    Icon(
        Icons.Filled.Done,
        contentDescription = "Save Contact"
    )
}
```

  - The `popBackStack("list")` command will remove from the back stack all the screens stored on the path from the *List Screen* up to the current screen
    - The `inclusive` parameter set to `false` is used to stay on the *List Screen* without returning to the previous screen of *List Screen*

List Screen

- Create the `ListScreen` composable to list all the contacts through cards
- The composable function should receive as parameters:
  - the list of contacts
  - the current expand mode (if the cards are to be shown expanded or collapsed)
  - a *lambda expression* with the code to execute when a contact card is clicked

```
@Composable
fun ListScreen(
    contacts : List<Contact>,
    showExpanded : Boolean = false,
    onSelectContact : (Contact) -> Unit = {},
    modifier: Modifier = Modifier
) {
   val formatter=SimpleDateFormat("yyyy.MM.dd",Locale.getDefault())
}
```

List Screen

- Use a `LazyColumn` to display the contacts
  - Each contact card will be managed by an independent composable

```
LazyColumn(
    modifier = modifier.fillMaxSize(),
) {
    items(
        contacts,
        key = { contact -> contact.hashCode() }
    ) { contact ->
        ContactCard(
            contact = contact,
            showExpanded = showExpanded,
            onSelectContact = onSelectContact,
        )

    }
}
```

List Screen – Card

- The `Card` composable is a basic card component that provides a simple container with optional elevation
  - The `ElevatedCard` is a variant of `Card` that provides a built-in, slightly more prominent shadow effect

```
Card(
    modifier = modifier
        .fillMaxWidth()
        .padding(8.dp),
    elevation = CardDefaults.cardElevation(4.dp),
    colors = CardDefaults.cardColors(
        containerColor = Color(255,224,192)
    ),
    onClick = { onSelectContact(contact) }
) {
    Column(
        modifier = Modifier.padding(16.dp),
        verticalArrangement = Arrangement.SpaceBetween
    ) {
        Text(text = contact.name, fontSize = 20.sp)
        if (showExpanded) {
                /* show all contact's information - see next slide */
        }
    }
}
```

List Screen – Card

```
if (showExpanded) {
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
        Text(text = contact.email, fontSize = 14.sp)
        Text(text = contact.phone, fontSize = 14.sp)
    }
    contact.birthday?.let { birthday ->
        Text(
            modifier = Modifier.fillMaxWidth(),
            textAlign = TextAlign.Center,
            text = formatter.format(birthday),
            fontSize = 14.sp
        )
    } ?: Text(modifier = Modifier.fillMaxWidth(),
        textAlign = TextAlign.Center,
        text = "----.--.--",
        fontSize = 14.sp)
}
```

Main Screen → List Screen

- In the `MainScreen`'s `NavHost` call the `ListScreen` composable function with the correct parameters

```
NavHost(
    navController = navController,
    startDestination = "list",
    modifier = Modifier.padding(innerPadding)
) {
    composable("list") {
        ListScreen(
            contacts = viewModel.contactsList.getContacts(),
            showExpanded = showExpanded,
            onSelectContact = { contact ->
                Log.d("MainScreen",
                      "onSelectContact:${contact.name}")
            }
        )
    }
    ...
}
```

Test

- Test the application by creating some fake contacts in the `ContactsApp` object
  - Include a method in the model to allow resetting the contact list, so fake contacts are not saved

View Model – Select contact

- Add a method to the View Model to select a contact
  - The selected contact should be stored internally so that it can be shared by multiple application screens
- Call this method in the context of the `onSelectContact` code defined in the *Main Screen*

  - *ContactsViewModel*:

```
var currentContact : Contact? = null

fun selectContact(contact : Contact) {
    currentContact = contact
}
```

  - *MainScreen*:

```
composable("list") {
    ListScreen(
        contacts = viewModel.contactsList.getContacts(),
        showExpanded = showExpanded,
        onSelectContact = { contact ->
            viewModel.selectContact(contact)
            navController.navigate("show")
        }
    )
}
```

Show Screen

- Create the `ShowScreen` composable function to display the information related to the selected contact
  - This composable should receive as parameter the contact to display
- Call this composable function in the "*show*" branch of the `NavHost`

```
@Composable
fun ShowScreen(
    contact: Contact,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxSize().padding(16.dp)
    ) {  }
}
```

- *MainScreen*:

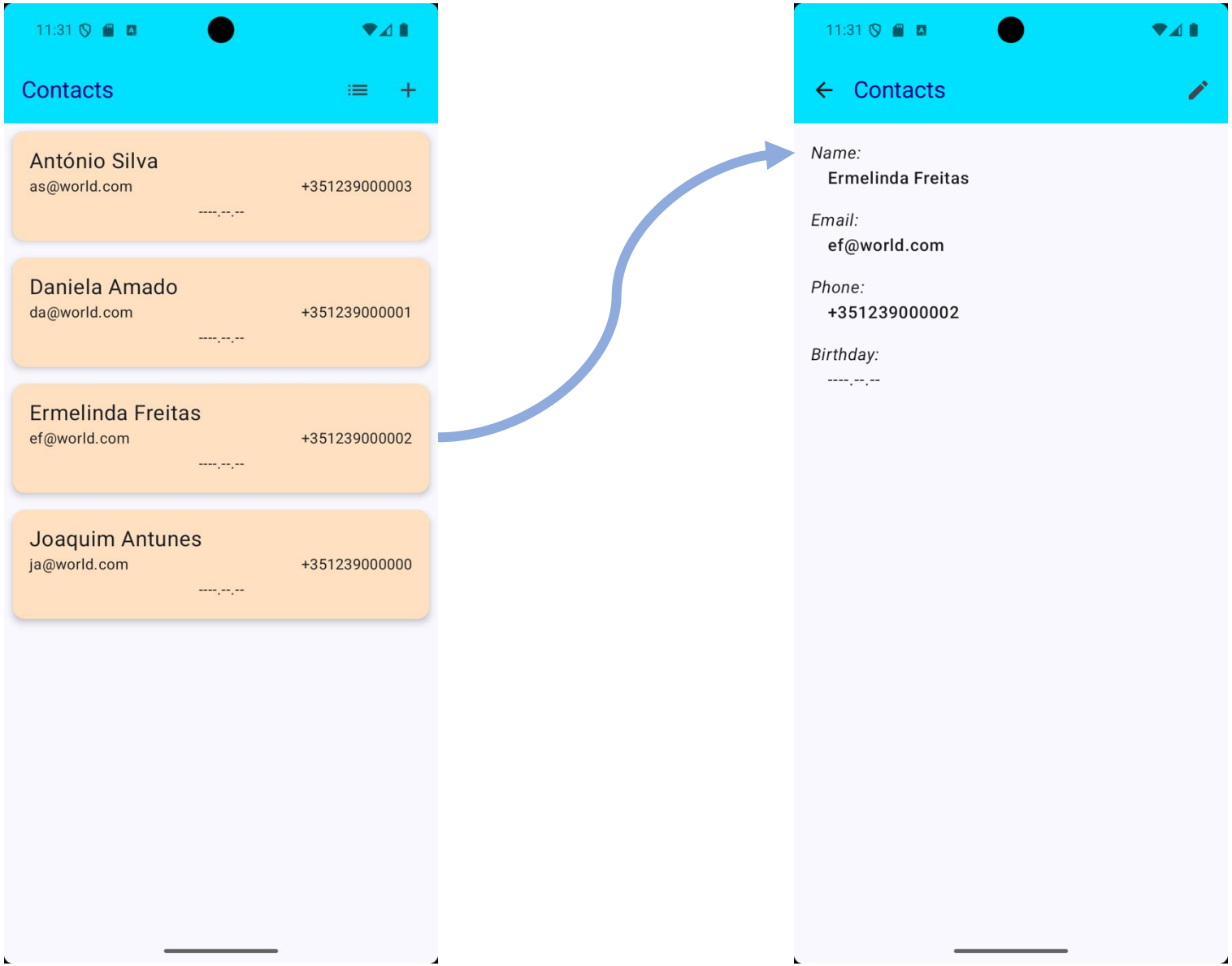
```
    composable("show") {
        viewModel.currentContact?.let { contact ->
            ShowScreen(contact = contact)
        } ?: Text("No contact selected")
    }
```

Show screen

- Display all contact information, including:
  - *Name*
  - *E-mail*
  - *Phone*
  - *Birthday*, if defined (otherwise, display "`----.--.--`")

- Format
  - Field name: *italic*, font size 16.sp
  - Field data: *bold* (*medium*), font size 16.sp, indented by 16.dp

Application screens



Edit Screen

- The *Edit Screen* will be used to enter the information for a new contact but also to edit an existing one
- The user can enter all the contact information but, if the back button is pressed, the information changed will not be saved
  - When creating a new contact that operation will be aborted (no new contact will be created)
- Therefore, instead of using the `currentContact:Contact`, that already exists in the *View Model*, independent temporary variables should be created to store the values
  - The new data is only stored if the edit operation is confirmed
    - For an existing contact the temporary information will override the existent
    - For a new contact, a new `Contact` object must be created with the new information entered by the user

TextField and OutlinedTextField

- To use `TextField` or `OutlinedTextField`, state variables of `String` type should be created to store the values

```
var name = remember {mutableStateOf("")}

OutlinedTextField(
    value = name.value,
    isError = name.value.isEmpty(),
    label = {
        Text("Name:")
    },
    onValueChange = { newText ->
        name.value = newText
    },
    modifier = Modifier
        .fillMaxWidth()
)
```

DatePicker

- For the `DatePicker` a state variable of the type `DatePickerState` is needed

```
var datePickerState = rememberDatePickerState()

DatePicker(
    state = datePickerState,
)
```

View Model – variables MutableState

- To ensure compatibility between the creation of temporary storage variables for contact fields subject to editing and the `State` variables required for the `TextField` and `DatePicker` composables, `MutableState` variables will be created in the View Model

```
val name = mutableStateOf("")
val phone = mutableStateOf("")
val email = mutableStateOf("")
val birthdayDPState = DatePickerState(
    Locale.getDefault(),
    Date().time
)
```

View Model – create and select

- Complete the previously created method for selecting a contact so that the new variables are updated with the selected contact's information

- Add a new method to create a contact:
  - Set the `currentContact` variable to `null` to represent that no contact is selected
  - Reset the new state variables to default values for each field

View Model – create and select

```
val name = mutableStateOf("")
val phone = mutableStateOf("")
val email = mutableStateOf("")
val birthdayDPState = DatePickerState(Locale.getDefault(), Date().time)

fun createContact() {
    currentContact = null
    name.value = ""
    phone.value = ""
    email.value = ""
    birthdayDPState.selectedDateMillis = 0
}

fun selectContact(contact : Contact) {
    currentContact = contact
    name.value = contact.name
    phone.value = contact.phone
    email.value = contact.email
    birthdayDPState.selectedDateMillis = contact.birthday?.time ?: 0
}
```

Edit Screen

- Redefine the parameters of the `EditScreen` function to receive references to the state variables for each field
  - Update its usage in the `NavHost` accordingly

```
@Composable
fun EditScreen(
    name : MutableState<String>,
    email : MutableState<String>,
    phone : MutableState<String>,
    birthday : DatePickerState,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp),
    ) {

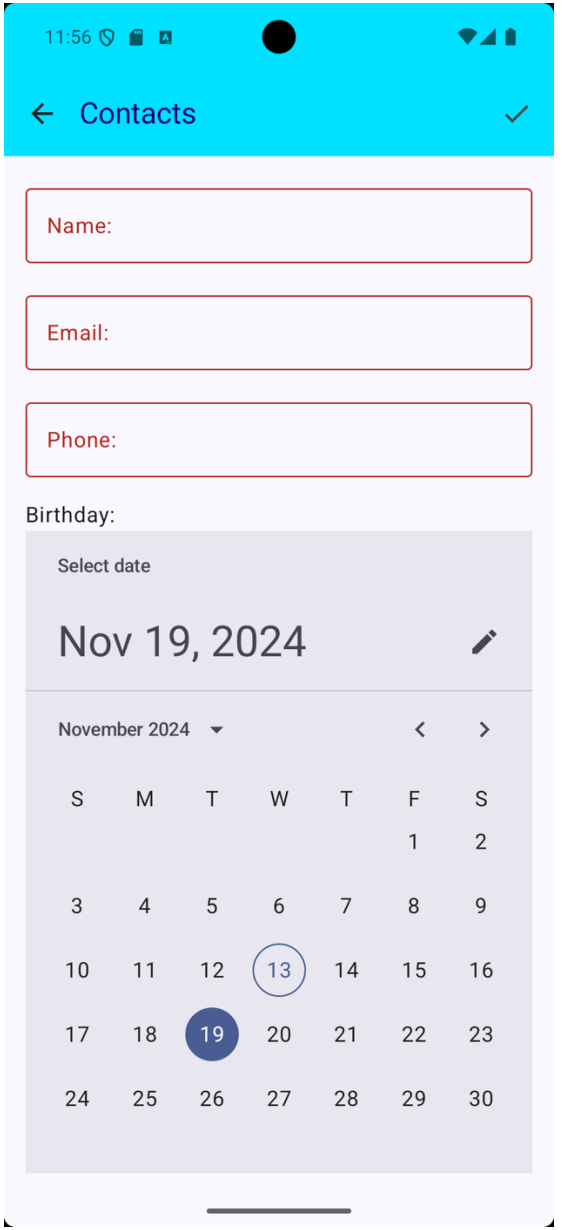
    }
}
```

Edit Screen

- Include `OutlinedTextField` and `DatePicker` composables to enter the data for each field

- Call the `EditScreen` composable function in the "*edit*" branch of the `NavHost`

```
composable("edit") {
    EditScreen(
        name     = viewModel.name,
        email    = viewModel.email,
        phone    = viewModel.phone,
        birthday = viewModel.birthdayDPState
    )
}
```



New contact action

- Ensure that when the "*Add*" button is pressed to create a new contact, the "*edit*" screen has empty fields

```
IconButton(
    onClick = {
        viewModel.createContact()
        navController.navigate("edit")
    }
) {
    Icon(
        Icons.Filled.Add,
        contentDescription = "Add Contact"
    )
}
```

Save contact

- If the *back* is pressed the changes performed must be ignored
- However, if the "done/save" button is pressed the contact must be created (if it is a new contact) or updated
  - To do that, in the respective action button in the `MainScreen`, a call to the appropriate view model method should be performed

```
IconButton(
    onClick = {
        if (viewModel.saveContact()) {
            navController.popBackStack("list", inclusive=false)
        }
    }
) {
    Icon(
        Icons.Filled.Done,
        contentDescription = "Save Contact"
    )
}
```

Save contact – View Model

```
fun saveContact() : Boolean {
    if (name.value.isEmpty() || email.value.isEmpty() || phone.value.isEmpty()) {
        return false
    }
    currentContact?.let { contact ->
        contact.name = name.value
        contact.phone = phone.value
        contact.email = email.value
        contact.birthday = birthdayDPState.selectedDateMillis?.let { Date(it) }
    } ?: contactsList.addContact(
        Contact(
            name.value,
            phone.value,
            email.value,
            birthdayDPState.selectedDateMillis?.let { Date(it) }
        )
    )
    return true
}
```

Model evolution

- Update the model to include a new field in the `Contact` object for storing a contact's picture
  - The new field should be of type `String?`, storing the path to an image in the application's internal storage

```
class Contact(
    var name     : String,
    var email    : String,
    var phone    : String,
    var birthday : Date?   = null,
    var picture  : String? = null,
): Serializable
```

- Apply this change throughout the application code

Display an image

- It is important to take some precautions when working with images
- There are memory limitations associated with image processing that can lead to unexpected results in applications
  - Loading images that are several megabytes in size into memory doesn't make sense if they are only being displayed in a small section of the screen
- To address this, image sampling techniques are often used to minimize such issues
  - For instance, the `BitmapFactory` class provides methods and configurations that assist with this task
- To help developers surpass these problems, auxiliary libraries have long been used to aid in these tasks
  - Examples include *Picasso*, *Glide*, and, more recently, *Coil*.

Coil library

- Include the *Coil* libraries in the module `build.gradle`:

```
implementation("io.coil-kt.coil3:coil-compose:3.0.2")
implementation("io.coil-kt.coil3:coil-network-okhttp:3.0.2")
```

- Usage examples:

```
AsyncImage(
    model = "https://static-00.iconduck.com/assets.00/avatar-
icon-512x512-gu21ei4u.png",
    contentDescription = "Avatar Icon",
)

AsyncImage(
    model = R.drawable.avatar_icon,
    contentScale = ContentScale.Crop,
    contentDescription = "Avatar Icon",
    modifier = Modifier
        .fillMaxWidth(0.5f)
        .aspectRatio(1f)
        .clip(CircleShape)
        .align(Alignment.CenterHorizontally)
)
```

Show contact's picture

- Add the contact's picture to the "Show" screen
  - If the contact doesn't have a defined picture, display an image stored in the `drawable` resource folder



Edit contact's picture

- Add a button at the bottom of the `EditScreen` to select a picture for the contact being edited

```
Button( onClick = {} ) {
    Text("Select picture")
}
```

- Execute the code
- As can be observed, the button doesn’t appear!
  - The screen doesn't have enough space to display all the composables
- Solution: Add scrolling capabilities to the column that contains the screen's content

```
val scrollState = rememberScrollState()
Column(
    modifier = modifier
        .fillMaxSize()
        .verticalScroll(scrollState)
        .padding(16.dp),
) { ... }
```

Edit contact's picture

- Display the picture if it exists

```
Spacer(Modifier.height(16.dp))
picture.value?.let { path ->
    AsyncImage(
        model = path,
        contentDescription = "Contact's picture",
        modifier = Modifier
            .fillMaxWidth(0.5f)
            .align(Alignment.CenterHorizontally)
    )
}
```

Access special features – permissions

- To minimize unauthorized access to user information and certain device capabilities by an application, the *Android system* requires:
  - Declaration of access permission in the application manifest file
    - From API 23 onwards, it also became mandatory to request permissions at *runtime*, with the presentation of a dialog window with an explicit authorization request from the user
  - Implementation of specific protocols for accessing information or granting permission to access internal information

Permissions

- Include permissions in the manifest file

  `<uses-permission android:name="...." />`

- The application's need for special device features should be declared using the XML structure: `<uses-feature ... />`

- Include the following permissions in the manifest:
  - Note: only include the permissions that are genuinely required to support the application's functionalities

```
<uses-feature
        android:name="android.hardware.camera"
        android:required="false" />

<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
    android:maxSdkVersion="32" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="32" tools:ignore="ScopedStorage" />
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>
<uses-permission android:name="android.permission.READ_MEDIA_VISUAL_USER_SELECTED"/>
<uses-permission android:name="android.permission.CAMERA"/>
```

Permissions – runtime authorization

- For systems with API23 or later, permissions must also be requested at runtime

```
if (checkSelfPermission(android.Manifest.permission.CAMERA)!=PackageManager.PERMISSION_GRANTED) {
    askSinglePermission.launch(android.Manifest.permission.CAMERA)
}
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
    if (checkSelfPermission(android.Manifest.permission.READ_MEDIA_IMAGES) !=
                                                            PackageManager.PERMISSION_GRANTED) {
        askSinglePermission.launch(android.Manifest.permission.READ_MEDIA_IMAGES)
    }
} else if (
        checkSelfPermission(android.Manifest.permission.READ_EXTERNAL_STORAGE) !=
                                                            PackageManager.PERMISSION_GRANTED ||
        checkSelfPermission(android.Manifest.permission.WRITE_EXTERNAL_STORAGE) !=
                                                            PackageManager.PERMISSION_GRANTED) {
            askMultiplePermissions.launch(
                arrayOf( android.Manifest.permission.READ_EXTERNAL_STORAGE,
                         android.Manifest.permission.WRITE_EXTERNAL_STORAGE )
            )
}

private val askSinglePermission = registerForActivityResult(
    ActivityResultContracts.RequestPermission()
) { granted -> /* TODO */}

private val askMultiplePermissions = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { map -> /* TODO */}
```

FileUtils

- Retrieve the `FileUtils` class from:
  - https://gist.github.com/ansisec

… which includes a set of methods for:

- Creating files with unique names
- Creating files from URIs

GetContent

- For the specific case of choosing information from another entity (e.g., the Android system), the `GetContent` contract can be used
  - In case of success, the `GetContent` contract returns a URI for the selected content (e.g, an image or a generic file). Otherwise, it returns `null`
- For example, to request an image resource:

```
getContent.launch("image/*")
```

  - Where:

```
val getContent = registerForActivityResult(
    ActivityResultContracts.GetContent()
) { uri ->
      imagePath = uri?.let {FileUtils.createFileFromUri(context,it)}
}
```

  - In *Jetpack Compose*, it is possible to use:

```
val getContent = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.GetContent(),
    onResult = { uri ->
       imagePath = uri?.let {FileUtils.createFileFromUri(context,it)}
    }
)
```

PickVisualMedia contract

- From API 30 (Android 11), it is possible to use the `PickVisualMedia` or `PickMultipleVisualMedia` contracts to select image or video resources
  - A specialized UI (half-window dialog) is presented for easy selection of images or videos
  - Access permissions are given per selected resource
    - No prior permission requests are required for this type of resource

PickVisualMedia contract

- From `https://developer.android.com/training/data-storage/shared/photopicker`:
  - *Older devices that run Android 4.4 (API level 19) through Android 10 (API level 29) and Android Go devices running Android 11 or 12 that support Google Play services can install a backported version of the photo picker. To enable the automatic installation of the backported photo picker module through Google Play services, add the following entry to the <application> tag in your app's manifest file:*

```
<!-- Trigger Google Play services to install the backported photo picker module. -->
<service android:name="com.google.android.gms.metadata.ModuleDependencies"
         android:enabled="false"
         android:exported="false"
         tools:ignore="MissingClass">
    <intent-filter>
       <action android:name="com.google.android.gms.metadata.MODULE_DEPENDENCIES"/>
    </intent-filter>
    <meta-data android:name="photopicker_activity:0:required" android:value=""/>
</service>
```

PickVisualMedia contract

- Add the `PickVisualMedia` contract to the `EditScreen` to select the contact's picture

```
val pickImage = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.PickVisualMedia(),
    onResult = { uri ->
        picture.value = uri?.let { FileUtils.createFileFromUri(context,it) }
    }
)

...

Button(
    onClick = {pickImage.launch(
                  PickVisualMediaRequest(
                       ActivityResultContracts.PickVisualMedia.ImageOnly ) ) },
    modifier = Modifier.align(Alignment.CenterHorizontally),
) {
    Text("Select picture")
}
```

Take picture

- When an application needs to capture an image, there are two main approaches:
  - Using an existing camera application (implicit *Intent*)
    - The application requests the system to open an external camera application to perform the task
    - Advantages:
      - Simpler implementation
      - No need to handle complex camera APIs directly
    - This technique will be used in the application developed during this exercise
  - Embedding the camera functionality (custom integration)
    - The application integrates the camera directly, allowing greater control over its features and behavior
    - Advantages:
      - Full customization of the camera interface and functionality
    - Implementation:
      - Use the *CameraX* Jetpack library
        - `https://developer.android.com/media/camera/camerax`

Take a picture – implicit Intent

- The process of taking photographs using the contract model can be carried out as follows:
  - Use the implicit intent `MediaStore.ACTION_IMAGE_CAPTURE` to start the camera activity

```
imagePath = FileUtils.getTempFilename(context)
fileUri = Uri.fromFile( File(imagePath) )
takePicture.launch(
    Intent(MediaStore.ACTION_IMAGE_CAPTURE).apply {
        putExtra( MediaStore.EXTRA_OUTPUT, fileUri )
    }
)
```

  - Use the `rememberLauncherForActivityResult` API to manage the result of the launched activity:

```
val takePicture = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.StartActivityForResult()
) { activityResult  ->
    if (activityResult.resultCode == Activity.RESULT_OK) {
        picture.value = FileUtils.copyFile(context,imagePath)
    }
}
```

Protections – FileUriExposed

- For systems with API24 and later, it is no longer possible to directly expose paths to internal application files

- For this exposure of internal data to be carried out consciously, the mechanism called `FileProvider` must be used
  - https://developer.android.com/training/secure-file-sharing/setup-sharing

FileProvider

- To respect the application's file sharing protocol with other applications, the `FileProvider` mechanisms to register the locations and files that other applications can access must be used
  - Create a *XML file* with details of the resources (`paths`) for which access is granted
  - Add the *provider* `androidx.core.content.FileProvider` to the application (manifest file)
  - Get a *URI* for the file using the method: `FileProvider.getUriForFile(…)`
    - This URI will be provided to applications that need access to the resources/files concerned

FileProvider – Example

- res/xml/file_paths.xml

```
<?xml version="1.0" encoding="utf-8"?>
<paths xmlns:android="http://schemas.android.com/apk/res/android">
    <external-files-path name="my_images" path="Pictures" />
    <external-cache-path name="my_cache"  path="." />
</paths>
```

- AndroidManifest.xml

```
...
<application ...>
    ...
    <provider
        android:name="androidx.core.content.FileProvider"
        android:authorities="pt.isec.amov.amovcontacts.android.fileprovider"
        android:exported="false" android:grantUriPermissions="true">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_paths" />
    </provider>
    ...
</application>
```

FileProvider

- Obtaining the secure URI for the file so that it will be available to other applications
  - Example:

```
...
imagePath = getTempFilename(context)
fileUri = FileProvider.getUriForFile(
             context,
             "pt.isec.amov.amovcontacts.android.fileprovider",
             File(imagePath)
          )
...
```

Take a photo – TakePicture contract

- Through the `TakePicture` contract model, the process of taking photos can be carried out as follows:
  - Launch the camera activity

```
imagePath = getTempFilename(context)
fileUri = FileProvider.getUriForFile(
            context,
            "pt.isec.amov.amovcontacts.android.fileprovider",
            File(imagePath)
          )
takePicture.launch(fileUri)
```

  - Handle the result:

```
var takePicture = registerForActivityResult(
    ActivityResultContracts.TakePicture()
) { success ->
    if (success) {
        picture.value = FileUtils.copyFile(context,imagePath)
    }
}
```

Contact – meeting locations

- Add functionality for storing meeting locations in the `Contact` class

- Define a `data class` for geolocation and date

```
data class MeetingPoint(
    val latitude: Double,
    val longitude: Double,
    val date: Date
) : Serializable
```

- Add a `MutableList` of `MeetingPoint` to the `Contact` class

```
val history:MutableList<MeetingPoint> = mutableListOf()
```

Contact – meeting locations

- Add methods to manage meeting locations

```
fun addMeetingPoint(
      latitude: Double,
      longitude: Double,
      date: Date
) {
    history.add(MeetingPoint(latitude, longitude, date))
}

fun addMeetingPoint(
      meetingPoint: MeetingPoint
) {
    history.add(meetingPoint)
}

fun getMeetingPoints(): List<MeetingPoint> {
    return history.toList()
}
```

Android

User Location (device geolocation)

Location services

- In most contemporary mobile devices, various technologies enable equipment location, including:
  - GPS (via satellite constellations)
  - GSM triangulation (technically "trilateration")
  - WiFi-based positioning
  - Other systems...

- These technologies provide access to detailed location information such as:
  - Latitude
  - Longitude
  - Altitude
  - Accuracy
  - ...

Location Manager

- Location information can be obtained using the *Android manager* called "Location Manager"

```
val locationManager = getSystemService(LOCATION_SERVICE)
                                                    as LocationManager
// alternative:
val locationManager : LocationManager by lazy {
        getSystemService(LOCATION_SERVICE) as LocationManager
}
```

- The `LocationManager` allows an application to request location updates from various providers

```
locationManager.requestLocationUpdates(
        provider: String,
        minTime: Long,
        minDistance: Float,
        listener: LocationListener
)
```

- When location updates are no longer needed, the listener should be cancelled to conserve resources

```
locationManager.removeUpdates(listener)
```

Location Manager

- `requestLocationUpdates` parameters
  - *Provider* – source of information
    - `GPS_PROVIDER`
    - `NETWORK_PROVIDER`
    - `PASSIVE_PROVIDER`
    - `FUSED_PROVIDER` (API 31)
  - *minTime* – minimum interval (*ms*) between location updates
  - *minDistance* – minimum distance (*m*) between location updates
  - *Listener* – listener that will be called whenever there is an information update
    - Alternatives:
      - Implementing it in an existing class
      - Defining a new class for this purpose
      - Defining an anonymous class
    - It is necessary to define the abstract method `onLocationChanged` of the interface `LocationListener`
    - The `requestLocationUpdates` method can be used to express interest in different types of locations and, eventually, process them using the same *listener*

Permissions

- The application must request permissions to access location methods
  - Include `<uses-permission … />` structures in the manifest file
- To get approximate coordinates

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

- To get more accurate coordinates (inc. satellite)

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

- To obtain coordinates in background (API 29+)

```
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
```

Permissions

- Request permissions at runtime

```
if (checkSelfPermission(android.Manifest.permission.ACCESS_COARSE_LOCATION)
                                                != PackageManager.PERMISSION_GRANTED ||
    checkSelfPermission(android.Manifest.permission.ACCESS_FINE_LOCATION)
                                                   != PackageManager.PERMISSION_GRANTED
) {
    askLocationPermissions.launch(
        arrayOf(
            android.Manifest.permission.ACCESS_COARSE_LOCATION,
            android.Manifest.permission.ACCESS_FINE_LOCATION
        )
    )
}

...

private val askLocationPermissions = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { map -> viewModel.hasLocationPermission =
             map[android.Manifest.permission.ACCESS_COARSE_LOCATION] == true ||
             map[android.Manifest.permission.ACCESS_FINE_LOCATION] == true
}
```

Location Services

- To obtain the location in a more effective way, a set of objects available in the `play-services-location` library can be used
  - `LocationServices`, `LocationRequest`, …

```
implementation("com.google.android.gms:play-services-location:21.3.0")
```

Energy efficiency vs location accuracy

- An advantage that can be obtained from the new objects is, for example, better battery management
- The use of location systems can have a significant impact on the energy efficiency of a device
  - The most appropriate methods should be chosen for the application objectives
    - *Is it necessary to obtain a highly accurate location to obtain the weather forecast?*

Create a location configuration

- Parameters to take into account in the `LocationRequest`
  - Accuracy of information
    - `setPriority(…)`
      - `Priority.PRIORITY_BALANCED_POWER_ACCURACY` (100m)
      - `Priority.PRIORITY_HIGH_ACCURACY`
      - `Priority.PRIORITY_LOW_POWER` (10km)
      - `Priority.PRIORITY_NO_POWER`
  - Frequency with which it is received
    - `setIntervalMillis(Long)`
    - `setMinUpdateIntervalMillis(Long)`
    - `setMaxUpdateDelayMillis(Long)`
    - `setMinUpdateDistanceMeters(float)`
    - …

LocationRequest

```
private val locationProvider: FusedLocationProviderClient by lazy {
        LocationServices.getFusedLocationProviderClient(this)
}
...
locationProvider.lastLocation
    .addOnSuccessListener { location ->
        if (location == null) return@addOnSuccessListener
        // proc. location
    }
...
val locationRequest =
    LocationRequest.Builder(PRIORITY_HIGH_ACCURACY, 2000)
        //.setMinUpdateDistanceMeters(100f)
        //.setMinUpdateIntervalMillis(1000)
        //.setMaxUpdateDelayMillis(5000)
        //.setPriority(PRIORITY_HIGH_ACCURACY)
        //.setIntervalMillis(2000)
        //.setMaxUpdates(10)
        .build()

locationProvider.requestLocationUpdates(locationRequest,locationCallback, Looper.getMainLooper())
...
private val locationCallback = object : LocationCallback() {
    override fun onLocationResult(result: LocationResult) {
        super.onLocationResult(result)
        result.locations.forEach { location ->
            // proc. location
        }
    }
}
...
locationProvider.removeLocationUpdates(locationCallback)
```

Other location capabilities

- *GeoFencing*
  - https://developer.android.com/develop/sensors-and-location/location/geofencing

- Detection of user activity type
  - Walking, cycling, driving, ...

  - https://developer.android.com/develop/sensors-and-location/location/transitions

Contact app – meeting locations

- Import the necessary classes and interfaces from `https://gist.github.com/ansisec` to access device location
  - `interface LocationHandler`
    - Defines common methods for different location implementation classes
  - `class LocationManagerHandler`
    - Utility class to use the classic `LocationManager` as the source for location data
  - `class FusedLocationHandler`
    - Utility class to use *Fused Location* services from *Google Play Services* to obtain location data

Contact app – meeting locations

- To integrate location handling in the `ContactsApp` class, initialize the desired implementation of `LocationHandler` using *lazy* initialization

```
/*
val locationHandler : LocationHandler by lazy {
    val locationManager = getSystemService(LOCATION_SERVICE)
                                            as LocationManager
    LocationManagerHandler(locationManager)
}
*/

val locationHandler : LocationHandler by lazy {
    val locationProvider = LocationServices
                          .getFusedLocationProviderClient(this)
    FusedLocationHandler(locationProvider)
}
```

Contact app – meeting locations

- Inject the `LocationHandler` into the *View Model* to enable location-related functionalities:

```
class ContactsViewModelFactory(
    private val contactsList: ContactsList,
    private val locationHandler: LocationHandler
) : ViewModelProvider.Factory {
    @Suppress("UNCHECKED_CAST")
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        return ContactsViewModel(contactsList,locationHandler) as T
    }
}

class ContactsViewModel(
    val contactsList: ContactsList,
    private val locationHandler: LocationHandler
) : ViewModel() {...}

class MainActivity : ComponentActivity() {
    private val app by lazy { application as ContactsApp }
    private val viewModel: ContactsViewModel by viewModels {
       ContactsViewModelFactory(app.contactsList, app.locationHandler)
    }
    ...
}
```

Contact app – meeting locations

- Include necessary methods and fields in the *View Model* to manage location

```
var hasLocationPermission: Boolean = false

private val currentLocation = mutableStateOf(Location(null))

init {
    locationHandler.onLocation = { location ->
        currentLocation.value = location
    }
}

fun startLocationUpdates() {
    if (hasLocationPermission)
        locationHandler.startLocationUpdates()
}

fun stopLocationUpdates() {
    locationHandler.stopLocationUpdates()
}
```

Contact app – meeting locations

- Add a method to store the current contact's location

```
fun storeCurrentLocation() {
    if (!hasLocationPermission || currentContact==null)
        return
    currentContact!!.addMeetingPoint(
        currentLocation.value.latitude,
        currentLocation.value.longitude,
        Date()
    )
}
```

Contact app – meeting locations

- Add an action button when displaying the "*Show*" screen to store the current contact's location

```
fun MainScreen(...) {
    var showConfirmationDialog by remember { mutableStateOf(false) }
    ...
    Scaffold(
        ...
        actions = {
            ...
               "show" -> {
                   IconButton(onClick = { showConfirmationDialog = true } ) {
                       Icon(Icons.Filled.LocationOn, contentDescription = "Store Location")
                   }
        ...
    ) { innerPadding ->
        NavHost(...) { ... }
        if (showConfirmationDialog) {
            ConfirmationDialog(
                actionDescription = "Store meeting location" ,
                onConfirm = {
                    showConfirmationDialog = false
                    viewModel.storeCurrentLocation()
                },
                onDismiss = { showConfirmationDialog = false }
            )
        }
    }
}
```

Confirmation Dialog

```
@Composable
fun ConfirmationDialog(
   actionDescription : String,
   onConfirm : () -> Unit,
   onDismiss : () -> Unit,
   modifier: Modifier = Modifier
) {
   AlertDialog(
      icon = {
         Icon(
            Icons.Default.Warning,
            contentDescription = "Confirmation dialog"
         )
      },
      title = { Text("Confirm operation") },
      text  = { Text("$actionDescription?") },
      onDismissRequest = onDismiss,
      confirmButton = { Button(onClick = onConfirm) { Text("Confirm")} },
      dismissButton = { Button(onClick = onDismiss) { Text("Cancel") } }
   )
}
```

Android

Maps

Google Maps

- To include Google Maps of the Google Cloud Platform (GCP) in an Android application, add *Google Play Services* to the project
  - Add the following line to the dependencies of the project's module, in the `build.gradle.kts` file of the module:

```
implementation("com.google.android.gms:play-services-maps:19.0.0")
```

Google Maps

- The manifest `<application … />` element in the manifest file must include version information

```
<meta-data
    android:name="com.google.android.gms.version"
    android:value="@integer/google_play_services_version"
/>
```

- *Note: this configuration is performed by default when the libraries mentioned before are included*

Google Maps

- The signing key used to create the *apk* of the application must be obtained for a full registration in GCP
  - In *debug mode*
    - Open a console in the `.android` directory in the user's profile directory
    - Run:

```
keytool -list -v -keystore debug.keystore
                -alias androiddebugkey -storepass android
                -keypass android
```

Note: the `keytool` is a Java SDK tool

Copy&paste : `keytool -list -v -keystore debug.keystore -alias androiddebugkey -storepass android -keypass android`

    - Copy the *SHA-1 key*
  - In *release mode*
    - The specific certificate created to sign the application to be distributed through the *PlayStore* must be used

Google Maps

- Obtain a certificate and key
  - Access the *Google APIs Console* website *(in GCP)*
    https://console.cloud.google.com/
  - Create a new project
  - Enable the *Google Maps SDK for Android API*
  - Add *API Key* credentials
    - Copy the *API key* generated for use in the XML structure indicated below
  - Restrict *API key* to only be used by the application being developed
    - Open *API key* settings
    - Choose *Applications restrictions* → *Android apps*
    - Add a new item
    - Fill in the *Package Name* field
    - Fill in the *SHA-1* field

  - Copy the *API key* created and add the following element to the `<application … />` structure in manifest file:

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="<insert API_KEY>"/>
```

Google Maps

- Include network access permissions in the manifest file

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

  - *The current versions of the libraries already include these permissions*

  - If needed, include:

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

- The support for *OpenGL ES v2* must be required

```
<uses-feature android:glEsVersion ="0x00020000" android:required ="true"/>
```

  - *The current versions of the libraries already include this configuration*

Google Maps – XML layouts

- Add the *Google Maps* visual element (a *fragment)*

```
<fragment
          android:id="@+id/map"
          android:layout_width="match_parent"
          android:layout_height="match_parent"
          android:name="com.google.android.gms.maps.SupportMapFragment"/>
```

  - After inserting the fragment, the application can be tested

- In *runtime*, the reference to the object map can be obtained doing:

```
(supportFragmentManager.findFragmentById(R.id.map) as?
SupportMapFragment)?.getMapAsync(<OnMapReadyCallback>)
```

MapFragment – XML Layouts

- The previous *slide* showed the inclusion of the fragment through the layout *XML file*. To add it at *runtime* do:

```
var mMapFragment = SupportMapFragment.newInstance()
val fragmentTransaction: FragmentTransaction =
                        supportFragmentManager.beginTransaction()
fragmentTransaction.add(R.id.my_container, mMapFragment)
fragmentTransaction.commit()
```

Example settings – XML Layouts

```
class MainActivity : AppCompatActivity(),OnMapReadyCallback {
  companion object {
    private val ISEC = LatLng(40.1925, -8.4115)
    private val DEIS = LatLng(40.1925, -8.4128)
  }
...
    (supportFragmentManager.findFragmentById(R.id.map) as? SupportMapFragment)?.getMapAsync(this)
...
  @SuppressLint("MissingPermission")
  override fun onMapReady(map: GoogleMap) {
    map.isMyLocationEnabled = true
    map.mapType = GoogleMap.MAP_TYPE_HYBRID
    map.uiSettings.isCompassEnabled = true
    map.uiSettings.isZoomControlsEnabled = true
    map.uiSettings.isZoomGesturesEnabled = true
    val cp = CameraPosition.Builder().target(ISEC).zoom(17f)
            .bearing(0f).tilt(0f).build()
    map.animateCamera(CameraUpdateFactory.newCameraPosition(cp))
    map.addCircle(CircleOptions().center(ISEC).radius(150.0)
            .fillColor(Color.argb(128, 128, 128, 128))
            .strokeColor(Color.rgb(128, 0, 0)).strokeWidth(4f))
    val mo = MarkerOptions().position(ISEC).title("ISEC-IPC")
            .snippet("Instituto Superior de Engenharia de Coimbra")
    val isec = map.addMarker(mo)
    isec?.showInfoWindow()
    map.addMarker(MarkerOptions().position(DEIS).title("DEIS-ISEC"))
  }
}
```

Google Maps in Jetpack Compose

- Google provides a Composable to include maps in the context of applications that use Jetpack Compose
  - Add the following line to the dependencies of the project's module, in the `build.gradle.kts` file of the module:

```
implementation("com.google.maps.android:maps-compose:4.3.0")
```

- Example of use:

```
val deisisec = LatLng(40.1925, -8.4128)
val cameraPositionState = rememberCameraPositionState {
  position = CameraPosition.fromLatLngZoom(deisisec, 13f)
}
GoogleMap(
  modifier = Modifier.fillMaxSize(),
  cameraPositionState = cameraPositionState
) {
  Marker(
    state = MarkerState(position = deisisec),
    title = "DEIS-ISEC",
    snippet = "Dep. Informatics Engineering"
  )
}
```

Google Maps in Jetpack Compose

- Code to show a set of *Markers* ensuring that all markers are visible:

```
val positions = arrayOf(
    LatLng(40.1925, -8.4128), LatLng(39.1925, -8.4128),
    LatLng(41.1925, -8.4128), LatLng(40.1925, -7.4128),
    LatLng(40.1925, -9.4128),
)
val boundsBuilder = LatLngBounds.Builder()
positions.forEach { boundsBuilder.include(it) }
val bounds = boundsBuilder.build()

val cameraPositionState = rememberCameraPositionState {
    CameraPosition.fromLatLngZoom(positions[0], 13f)
}
LaunchedEffect(bounds) {
    cameraPositionState.animate(
        update = CameraUpdateFactory.newLatLngBounds(bounds,64),
        durationMs = 5000
    )
}
GoogleMap(
    modifier = Modifier.fillMaxSize(),
    cameraPositionState = cameraPositionState
) {
    positions.forEach { latLng ->
        Marker(state = MarkerState(position = latLng) )
    }
}
```

LaunchedEffect and other *Effect

- `LaunchedEffect`
  - Allows running some code in response to changes in the element defined by its *key* parameter

    ```
    LaunchedEffect(keyObject) {/*code*/}
    ```
  - The code runs within the context of a coroutine
  - Supports suspending functions
- `SideEffect`
  - Runs code during every recomposition

    ```
    SideEffect {/*code*/}
    ```
  - Does not support suspending functions
- `DisposableEffect`
  - Runs code in response to changes in the element defined by its *key* parameter, similar to `LaunchedEffect`
  - However, an `onDispose{}` method can be defined to clean up resources when the key changes or when the composable is removed from the composition

    ```
    DisposableEffect(keyObject) {
        // code
        onDispose {
            // cleanup
        }
    }
    ```
- `LaunchedEffect` and `DisposableEffect` can both have a *key* parameter of `Unit` to ensure the code runs only once

Contacts App – meeting locations

- Include a map with markers in the "show" screen to indicate where the contact was met

OpenStreetMap (OSM)

- Other map services can be used, some of which are free, such as *OpenStreetMap*

- *OpenStreetMap* provides an API that can be used on Android
  - The objects used to access OSM have the same or a very similar name to the one used by Google Maps
  - Add the following line to the dependencies of the project's module, in the `build.gradle.kts` file of the module:

```
implementation("org.osmdroid:osmdroid-android:6.1.20")
```

OpenStreetMap – XML layouts

- Add the visual *OpenStreetMap* element to the layout:

```
<org.osmdroid.views.MapView android:id="@+id/map"
                android:layout_width="fill_parent"
                android:layout_height="fill_parent" />
```

- Before inflating the layout, include the following line to initialize the *OSM* configuration:

```
Configuration.getInstance().load(this,
                        getSharedPreferences("osm", MODE_PRIVATE))
```

- Get the reference to `MapView`:

```
private lateinit var map : MapView
…
map = findViewById<MapView>(R.id.map)
map.setTileSource(TileSourceFactory.MAPNIK)
```

https://github.com/osmdroid/osmdroid/wiki/How-to-use-the-osmdroid-library-(Kotlin)

OSM@Jetpack Compose

- Although there are already some unofficial libraries available for using *OpenStreetMap* in *Jetpack Compose* applications, the integration of traditional Android Views through the `AndroidView` composable can be used

- Example of use:

```
val context = LocalContext.current
var geoPoint by remember{
        mutableStateOf(GeoPoint(40.1925, -8.4128))
}

val mapView = MapView(context).apply {
        setTileSource(TileSourceFactory.DEFAULT_TILE_SOURCE)
        setMultiTouchControls(true)
        controller.setCenter(geoPoint)
        controller.setZoom(17.0)
}

AndroidView(
        factory = { mapView },
        update = { view ->
             view.controller.setCenter(geoPoint)
        }
)
```

https://developer.android.com/jetpack/compose/migrate/interoperability-apis/views-in-compose

OpenStreetMap for Android Compose

- Dependencies

https://utsmannn.github.io/osm-android-compose/

```
implementation("org.osmdroid:osmdroid-android:6.1.20")
implementation("tech.utsmankece:osm-androd-compose:0.0.5")
```

- Usage:

```
val cameraState = rememberCameraState {
    geoPoint = GeoPoint(40.1925, -8.4128)
    zoom = 12.0 // optional, default is 5.0
}
val deisISEC = rememberMarkerState(
    geoPoint = GeoPoint(40.1925, -8.4128)
)
Box(
    modifier = Modifier.fillMaxSize().padding(16.dp),
) {
    OpenStreetMap(
        modifier = Modifier.fillMaxSize(),
        cameraState = cameraState
    ){
        Marker(state = deisISEC) // com.utsman.osmandcompose.Marker
    }
}
```

Portrait ⇔ Landscape

- So that the application can adapt to the screen orientation
  - Define a composable for each orientation
    - Normally:
      - Portrait mode is based on a Column
      - Landscape mode is based on a Row
  - Choose the composable to present based on the current screen's orientation

```
@Composable
fun ShowScreen(
    contact: Contact,
    modifier: Modifier = Modifier
) {
    if (LocalConfiguration.current.orientation ==
                                Configuration.ORIENTATION_LANDSCAPE) {
        ShowScreenLandscape(contact,modifier)
    } else {
        ShowScreenPortrait(contact,modifier)
    }
}
```